

Parcial I - Evidencias

Jeremy Osvaldo Trejos Murillo

CURSO

ESTRUCT. DE DATOS Y ALGORITMOS T

PROFESOR

Fabián Chinchilla Mayorga

Segundo cuatrimestre, año 2026

Universidad Central

Índice

Descripción General	3
Descripción del diseño lógico implementado para simular LIFO con listas	3
Capturas de pantalla.....	4

Descripción General

El presente proyecto consiste en el desarrollo de una aplicación en Java que simula un acumulador numérico interactivo con soporte para deshacer y rehacer operaciones. El usuario puede aplicar operaciones de suma y resta sobre un valor acumulado, y en cualquier momento puede revertir o volver a aplicar dichas operaciones mediante los mecanismos de Undo y Redo.

El programa está dividido en tres clases con responsabilidades bien definidas. La clase Operacion modela cada acción matemática, encapsulando el valor numérico y el tipo de operación, además de calcular su propia inversa. La clase Acumulador contiene la lógica principal del sistema, gestionando el total acumulado y los dos historiales mediante ArrayList. Finalmente, la clase Proyecto1 actúa como interfaz de usuario, presentando un menú interactivo a través de JOptionPane y dirigiendo cada acción al método correspondiente del acumulador.

Toda la información se gestiona en memoria dinámica sin ningún tipo de persistencia en disco. El sistema valida las entradas del usuario para evitar formatos incorrectos, accesos a listas vacías y cierres inesperados del programa al presionar el botón Cancelar.

Descripción del diseño lógico implementado para simular LIFO con listas

En este proyecto se utilizaron dos instancias de ArrayList para gestionar el historial de operaciones: undoStack y redoStack. Aunque ArrayList no es una pila por naturaleza, se simuló el comportamiento LIFO (Last-In, First-Out) controlando manualmente el punto de inserción y extracción.

Para simular el tope de la pila, todas las inserciones se realizan al final de la lista mediante el método add(), y todas las extracciones se realizan también desde el último elemento usando remove(size() - 1). De esta forma, el último elemento agregado siempre es el primero en salir, respetando el principio LIFO.

El flujo funciona de la siguiente manera:

Operar: cuando el usuario aplica una operación (+N o -N), esta se agrega al final del undoStack y el redoStack se vacía por completo, ya que una nueva operación invalida cualquier acción deshecha anteriormente.

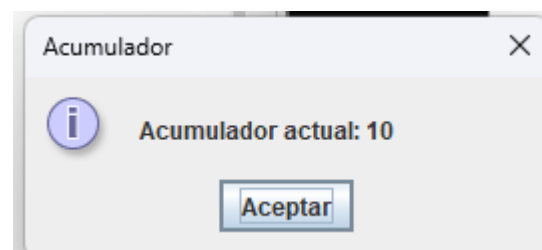
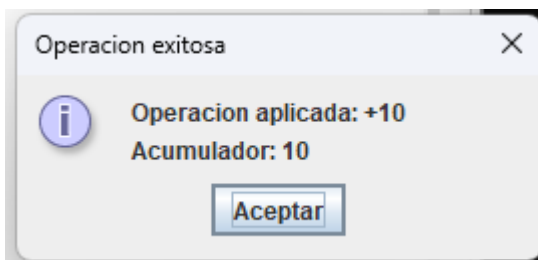
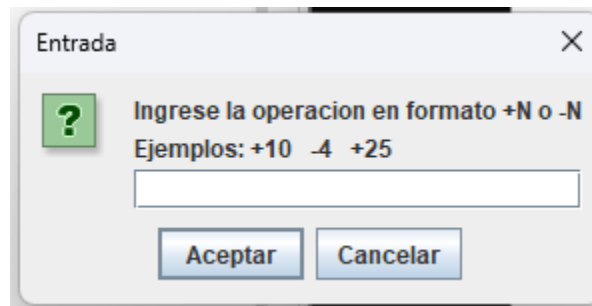
Undo: se extrae el último elemento del undoStack (el tope), se aplica su operación inversa al acumulador mediante el método inversa() de la clase Operacion, y la operación se traslada al final del redoStack.

Redo: se extrae el último elemento del redoStack (el tope), se vuelve a aplicar la operación al acumulador, y se devuelve al final del undoStack.

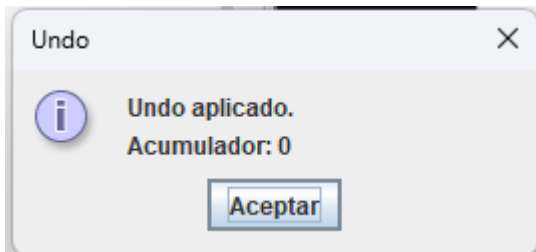
Este diseño garantiza que en ningún momento se accede a índices intermedios de las listas, manteniendo siempre el comportamiento de pila mediante la manipulación exclusiva del último elemento.

Capturas de pantalla

- Ingreso de operación y actualización del acumulador -



- Uso exitoso del mecanismo de Undo -



- Visualización del historial de los ArrayList o uso del Redo -

